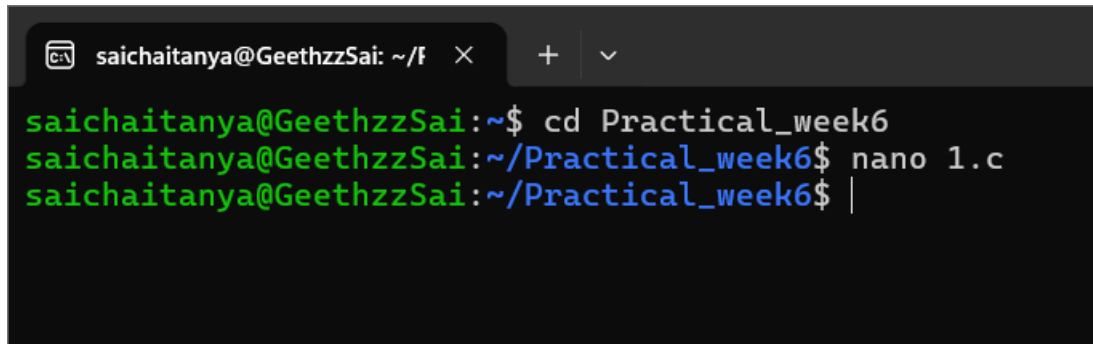


PRACTICAL WEEK 6

Task 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

1) Create a file



```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week6
saichaitanya@GeethzzSai:~/Practical_week6$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week6$ |
```

Then press enter to open command

2) write a code

```
saichaitanya@GeethzzSai: ~/f × + v
GNU nano 8.7.1 1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "server_fifo"
#define MAX 100

struct message
{
    char client_fifo[50];
    char message[MAX];
};

int main()
{
    int fd;
    struct message msg;
    char response[150];

    // Create named pipe
    mkfifo(SERVER_FIFO, 0666);

    printf("Server started. Waiting for clients...\n");

    while (1)
    {
        fd = open(SERVER_FIFO, O_RDONLY);

        if (fd == -1)
        {
            perror("open");
            exit(1);
        }
    }
}
```

```
saichaitanya@GeethzzSai: ~/f × + v
GNU nano 8.7.1 1.c
    fd = open(SERVER_FIFO, O_RDONLY);

    if (fd == -1)
    {
        perror("open");
        exit(1);
    }

    read(fd, &msg, sizeof(msg));
    close(fd);

    printf("\nClient message: %s", msg.message);

    // Process the message
    snprintf(response, sizeof(response),
              "Server received your message: %s",
              msg.message);

    // Open client's FIFO
    int client_fd = open(msg.client_fifo, O_WRONLY);

    if (client_fd == -1)
    {
        perror("client FIFO");
        continue;
    }

    write(client_fd, response, strlen(response) + 1);

    close(client_fd);

    printf("Response sent to client.\n");
}

return 0;
}
```

- 1) To save press control O
- 2) To come back to shell press control X
- 3) Then DO compilation

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week6
saichaitanya@GeethzzSai:~/Practical_week6$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week6$ gcc 1.c -o 1
```

OUTPUT:

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week6
saichaitanya@GeethzzSai:~/Practical_week6$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week6$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/Practical_week6$ ./1
Server started. Waiting for clients...
|
```

Task-2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSRI. Demonstrate asynchronous event handling and explain the role of signal handlers.

CODE:

```
saichaitanya@GeethzzSai: ~/f × + v
GNU nano 8.7.1 2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "server_fifo"
#define MAX 100

struct message
{
    char client_fifo[50];
    char message[MAX];
};

int main()
{
    int server_fd, client_fd;
    struct message msg;
    char response[150];

    // Create a unique FIFO for this client
    snprintf(msg.client_fifo, sizeof(msg.client_fifo),
             "client_%d_fifo", getpid());

    mkfifo(msg.client_fifo, 0666);

    printf("Client started. PID = %d\n", getpid());

    printf("Enter message: ");
    fgets(msg.message, MAX, stdin);

    // Open server FIFO
    server_fd = open(SERVER_FIFO, O_WRONLY);

    if (server_fd == -1)
```

```
// Open server FIFO
server_fd = open(SERVER_FIFO, O_WRONLY);

if (server_fd == -1)
{
    perror("server FIFO");
    unlink(msg.client_fifo);
    exit(1);
}

// Send message to server
write(server_fd, &msg, sizeof(msg));

close(server_fd);

// Wait for server response
client_fd = open(msg.client_fifo, O_RDONLY);

if (client_fd == -1)
{
    perror("client FIFO");
    unlink(msg.client_fifo);
    exit(1);
}

read(client_fd, response, sizeof(response));

printf("Server response: %s\n", response);

close(client_fd);

// Remove client FIFO
unlink(msg.client_fifo);

return 0;
```

```
}
```

OUTPUT:

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week6
saichaitanya@GeethzzSai:~/Practical_week6$ nano 2.c
saichaitanya@GeethzzSai:~/Practical_week6$ gcc 2.c -o 2
saichaitanya@GeethzzSai:~/Practical_week6$ ./2
Client started. PID = 708
Enter message: hello
Server response: Server received your message: hello

saichaitanya@GeethzzSai:~/Practical_week6$ |
```